

Lab 1

Your First Data Visualizations

YOUR NAME HERE

Overview

In this lab, you will practice adapting code from the lectures to work with a new dataset. You will:

1. Load and explore a new dataset
2. Create a bar chart by adapting code from Lesson 2.1
3. Create a histogram
4. Write brief interpretations of your visualizations
5. Render your document to PDF and submit

You are encouraged to have the lecture materials open while completing this lab.

Getting Started

Load the tidyverse package (make sure that you have it installed).

```
library(tidyverse)
```

The Data

Today we'll work with data from the Gapminder project, which tracks development indicators across countries. We have two files:

- `gapminder_summary.csv` - Average values by continent
- `gapminder_2007.csv` - Data for 142 individual countries in 2007

Part 1: Bar Chart (50 points)

Step 1: Load and Explore the Data (10 pts)

First, load the summary dataset and use `glimpse()` to see what variables are available.

```
# Load the data (this is done for you)
gapminder_summary <- read_csv("gapminder_summary.csv")
```

```
Rows: 5 Columns: 5
-- Column specification -----
Delimiter: ","
chr (1): continent
dbl (4): avg_life_exp, avg_gdp_cap, total_pop, n_countries

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Use glimpse() to explore the data
```

Question: What variables are in this dataset? How many columns are there? How many rows?

YOUR ANSWER HERE

Step 2: Create a Bar Chart (20 pts)

Using the bar chart code from **Lesson 2.1** as a template, create a bar chart using one of the variables in the dataframe. Your options are `avg_life_exp`, `avg_gdp_cap`, `total_pop`, or `n_countries`.

Think about what needs to change:

- What is the name of your data frame?
- What variable should go on the x-axis?
- What variable should go on the y-axis?
- What should your title and axis labels say?

```
# Write your bar chart code here
```

Step 3: Interpret Your Chart (10 pts)

Write 2-3 sentences describing what you see. Which continent has the highest value of the variable you chose to visualize? Which has the lowest?

YOUR INTERPRETATION HERE

Part 2: Histogram (50 pts)

Step 1: Load and Explore the Full Dataset (10 pts)

Now let's work with the full Gapminder dataset for 2007:

```
gapminder <- read_csv("gapminder_2007.csv")
```

```
Rows: 142 Columns: 6
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (2): country, continent
```

```
dbl (4): year, population, life_exp, gdp_cap
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Use glimpse() to explore this data
```

Describe these data? How many variables are there? How many countries are represented?

YOUR ANSWER HERE

Step 2: Create a Histogram (20 pts)

Using the histogram code from **Lesson 2.1** as a template, create a histogram showing the **distribution of life expectancy** across all countries.

Remember: histograms use `geom_histogram()` and only need an `x` variable in `aes()`.

```
# Write your histogram code here
```

Step 3: Interpret Your Histogram (10 pts)

Write 2-3 sentences about what you see. Is life expectancy spread evenly across countries, or are there clusters? What range do most countries fall into?

YOUR INTERPRETATION HERE

Render as PDF and Submit Your Work (20 pts)

1. Replace “YOUR NAME HERE” at the top with your actual name
2. Make sure all your code chunks run without errors
3. Click “Render” to create your PDF
4. Submit the PDF to Blackboard

Hints

Only look at these if you're stuck!

Hint 1 - Bar chart structure:

```
ggplot(data_name, aes(x = ___, y = ___)) +  
  geom_col() +  
  labs(title = "___", x = "___", y = "___")
```

Hint 2 - Histogram structure:

```
ggplot(data_name, aes(x = ___)) +  
  geom_histogram()
```

Hint 3 - Variable names matter: Make sure the variable names in your code match *exactly* what you saw in `glimpse()`. R is case-sensitive!